

Exact Single-Monomial High-Order Partial Derivatives via Waring Directional Schedules

Apolarity meets Taylor-mode AD

Apolarity contributors

June 2, 2026

Outline

- 1 The bottleneck
- 2 Two ingredients
- 3 Coefficient extraction view
- 4 The bridge: directional schedule $\Leftrightarrow$ Waring decomposition
- 5 Worked constructions
- 6 Why the schedule is faster than nested AD
- 7 The Taylor jet primitive
- 8 Algorithm and dispatch
- 9 Take-aways

What we want, and why it hurts

Many PINN / NN-VMC tasks need a *single* high-order mixed partial

$$\partial^\alpha u(x) = \partial_{i_1} \partial_{i_2} \cdots \partial_{i_p} u(x), \quad |\alpha| = p,$$

evaluated on a neural-network surrogate $u : \mathbb{R}^d \rightarrow \mathbb{R}$.

Examples:

- Cahn–Hilliard: $\partial_x^2 \partial_y^2 u$
- KdV: $\partial_x^3 u$
- Polyharmonic: $\Delta^k u = \sum_\alpha \partial^\alpha u$
- Beam equations: $\partial_x^4 u$

Naive cost. `torch.autograd.grad` with `create_graph=True` called p times: graph depth $\Theta(pL)$, memory $\Theta((p+1)L)$ tensors, and a heavy autograd-engine dispatch constant per op.

Concrete pain point

$p = 6$, depth-4 MLP, batch 8, hidden 64 on a Tesla T4:

- Each forward pass ≈ 1 ms.
- Six nested `autograd.grad` calls: graph carries six layers of higher-order tape; replay + dispatch dominate.
- Wall-clock per single $\partial^\alpha u$ call: **tens of milliseconds**.

In a PINN training loop, this single derivative call is invoked once per residual point per step. **It is the bottleneck.**

Goal of this work

A *deterministic, exact* backend for one fixed multi-index α that (i) avoids the autograd-graph chaining of nested `grad`, (ii) uses as few directional probes as the algebra allows, and (iii) propagates parameter gradients through training.

Two ingredients combined

Ingredient 1: directional view.

$$T_p(x; v) := \frac{1}{p!} D^p u(x)[v, \dots, v]$$

is the order- p Taylor coefficient of $g(\tau) = u(x + \tau v)$ at $\tau = 0$.

Instead of computing $\partial^\alpha u$ directly, evaluate T_p along a few directions $v_1, \dots, v_R$ and take a linear combination

$$\partial^\alpha u(x) = \sum_{r=1}^R c_r T_p(x; v_r).$$

Ingredient 2: choose directions wisely.

Each $T_p(x; v)$ still costs something. We want

R as small as possible.

Two ingredients combined

Ingredient 1: directional view.

$$T_p(x; v) := \frac{1}{p!} D^p u(x)[v, \dots, v]$$

is the order- p Taylor coefficient of $g(\tau) = u(x + \tau v)$ at $\tau = 0$.

Instead of computing $\partial^\alpha u$ directly, evaluate T_p along a few directions $v_1, \dots, v_R$ and take a linear combination

$$\partial^\alpha u(x) = \sum_{r=1}^R c_r T_p(x; v_r).$$

Ingredient 2: choose directions wisely.

Each $T_p(x; v)$ still costs something. We want

R as small as possible.

The minimum possible R is given by a classical theorem in algebraic geometry — the *monomial Waring rank* — and an explicit construction comes from *roots of unity*.

The generating polynomial Q_u

Plug $v = \sum_{j=0}^n z_j e_{ij}$ into T_p and expand:

$$Q_u(z) := T_p\left(x; \sum_j z_j e_{ij}\right) = \sum_{|\beta|=p} \frac{\partial^\beta u(x)}{\beta!} z^\beta. \quad (\heartsuit)$$

This is a degree- p homogeneous polynomial in z , whose coefficients are the p -th order partials of u .

The generating polynomial Q_u

Plug $v = \sum_{j=0}^n z_j e_{ij}$ into T_p and expand:

$$Q_u(z) := T_p\left(x; \sum_j z_j e_{ij}\right) = \sum_{|\beta|=p} \frac{\partial^\beta u(x)}{\beta!} z^\beta. \quad (\heartsuit)$$

This is a degree- p homogeneous polynomial in z , whose coefficients are the p -th order partials of u .

Coefficient extraction identity

$$\partial^\alpha u(x) = \alpha! [z^\alpha] Q_u(z).$$

The generating polynomial Q_u

Plug $v = \sum_{j=0}^n z_j e_{ij}$ into T_p and expand:

$$Q_u(z) := T_p\left(x; \sum_j z_j e_{ij}\right) = \sum_{|\beta|=p} \frac{\partial^\beta u(x)}{\beta!} z^\beta. \quad (\heartsuit)$$

This is a degree- p homogeneous polynomial in z , whose coefficients are the p -th order partials of u .

Coefficient extraction identity

$$\partial^\alpha u(x) = \alpha! [z^\alpha] Q_u(z).$$

So computing $\partial^\alpha u$ is the same as extracting one coefficient of a homogeneous polynomial. *Now we ask the algebra question:*

When can the linear functional “extract z^α coefficient” be written as a finite linear combination of point evaluations $Q \mapsto Q(v_r)$?

Worked example: u_{12} ($p = 2$, $\alpha = (1, 1)$)

Active variables x_1, x_2 . Direct expansion:

$$T_p(x; (z_0, z_1)) = \frac{u_{11}}{2} z_0^2 + u_{12} z_0 z_1 + \frac{u_{22}}{2} z_1^2.$$

Want $u_{12} = \sum_r c_r T_p(x; (a_r, b_r))$ for all u : three scalar conditions on coefficients,

$$\sum_r c_r a_r^2 = 0, \quad \sum_r c_r a_r b_r = 1, \quad \sum_r c_r b_r^2 = 0.$$

Worked example: u_{12} ($p = 2$, $\alpha = (1, 1)$)

Active variables x_1, x_2 . Direct expansion:

$$T_p(x; (z_0, z_1)) = \frac{u_{11}}{2} z_0^2 + u_{12} z_0 z_1 + \frac{u_{22}}{2} z_1^2.$$

Want $u_{12} = \sum_r c_r T_p(x; (a_r, b_r))$ for all u : three scalar conditions on coefficients,

$$\sum_r c_r a_r^2 = 0, \quad \sum_r c_r a_r b_r = 1, \quad \sum_r c_r b_r^2 = 0.$$

Repackage these conditions as one polynomial identity in dual variables w_0, w_1 :

$$\sum_r c_r (a_r w_0 + b_r w_1)^2 = \left(\sum_r c_r a_r^2 \right) w_0^2 + 2 \left(\sum_r c_r a_r b_r \right) w_0 w_1 + \left(\sum_r c_r b_r^2 \right) w_1^2 = 2 w_0 w_1.$$

Worked example: u_{12} ($p = 2$, $\alpha = (1, 1)$)

Active variables x_1, x_2 . Direct expansion:

$$T_p(x; (z_0, z_1)) = \frac{u_{11}}{2} z_0^2 + u_{12} z_0 z_1 + \frac{u_{22}}{2} z_1^2.$$

Want $u_{12} = \sum_r c_r T_p(x; (a_r, b_r))$ for all u : three scalar conditions on coefficients,

$$\sum_r c_r a_r^2 = 0, \quad \sum_r c_r a_r b_r = 1, \quad \sum_r c_r b_r^2 = 0.$$

Repackage these conditions as one polynomial identity in dual variables w_0, w_1 :

$$\sum_r c_r (a_r w_0 + b_r w_1)^2 = \left(\sum_r c_r a_r^2 \right) w_0^2 + 2 \left(\sum_r c_r a_r b_r \right) w_0 w_1 + \left(\sum_r c_r b_r^2 \right) w_1^2 = 2 w_0 w_1.$$

So the question becomes: "Write the monomial $2w_0w_1$ as a sum of squares of linear forms."

The bridge theorem

Theorem (Bridge)

Given $R \in \mathbb{N}$, $c_r \in \mathbb{K}$, $v_r \in V_{\text{active}}$, the following are equivalent:

- ① (directional schedule) $\partial^\alpha u(x) = \sum_{r=1}^R c_r T_p(x; v_r) \quad \forall u \in C_x^p.$
- ② (functional identity on Sym^p) $\alpha! [z^\alpha] Q = \sum_r c_r Q(v_r) \quad \forall Q \in \mathbb{K}[z]_p.$
- ③ (polynomial identity) $p! w^\alpha = \sum_r c_r (v_r \cdot w)^p \text{ in } \mathbb{K}[w]_p.$

The bridge theorem

Theorem (Bridge)

Given $R \in \mathbb{N}$, $c_r \in \mathbb{K}$, $v_r \in V_{\text{active}}$, the following are equivalent:

- ① (directional schedule) $\partial^\alpha u(x) = \sum_{r=1}^R c_r T_p(x; v_r) \quad \forall u \in C_x^p.$
- ② (functional identity on Sym^p) $\alpha! [z^\alpha] Q = \sum_r c_r Q(v_r) \quad \forall Q \in \mathbb{K}[z]_p.$
- ③ (polynomial identity) $p! w^\alpha = \sum_r c_r (v_r \cdot w)^p \text{ in } \mathbb{K}[w]_p.$

Statement (3) is exactly a *Waring decomposition* of the monomial w^α : writing it as a sum of p -th powers of linear forms.

Corollary. $R_K^*(\alpha) = R_K(w^\alpha)$ (the Waring rank). Algebra geometry tells us the minimum number of directions.

Two-step proof (sketch)

- ④ (1) $\Leftrightarrow$ (2): Lemma: $u \mapsto Q_u$ is surjective onto $\mathbb{K}[z]_p$ (just take u a polynomial). So “for all u ” becomes “for all Q ”.

Two-step proof (sketch)

- 1 (1) $\Leftrightarrow$ (2): Lemma: $u \mapsto Q_u$ is surjective onto $\mathbb{K}[z]_p$ (just take u a polynomial). So “for all u ” becomes “for all Q ”.
- 2 (2) $\Leftrightarrow$ (3): Test (2) on monomial basis $\{z^\beta\}$:

$$\alpha! \delta_{\alpha\beta} = \sum_r c_r v_r^\beta, \quad \forall |\beta| = p.$$

Multiply by $\binom{p}{\beta} w^\beta$ and sum:

$$\underbrace{\alpha! \binom{p}{\alpha} w^\alpha}_{=p! w^\alpha} = \sum_r c_r \underbrace{\sum_\beta \binom{p}{\beta} v_r^\beta w^\beta}_{=(v_r \cdot w)^p}.$$

Reverse direction reads off coefficients of w^β . $\square$

u_{12} : rank 2, polarization identity

Want $2w_0w_1$ as sum of squares.

$$2w_0w_1 = \frac{1}{2}(w_0 + w_1)^2 - \frac{1}{2}(w_0 - w_1)^2.$$

u_{12} : rank 2, polarization identity

Want $2w_0w_1$ as sum of squares.

$$2w_0w_1 = \frac{1}{2}(w_0 + w_1)^2 - \frac{1}{2}(w_0 - w_1)^2.$$

Read off:

$$v_1 = (1, 1), \quad c_1 = \frac{1}{2}; \quad v_2 = (1, -1), \quad c_2 = -\frac{1}{2}.$$

Translation:

$$u_{12} = \frac{1}{2} T_p(x; e_1 + e_2) - \frac{1}{2} T_p(x; e_1 - e_2).$$

u_{12} : rank 2, polarization identity

Want $2w_0w_1$ as sum of squares.

$$2w_0w_1 = \frac{1}{2}(w_0 + w_1)^2 - \frac{1}{2}(w_0 - w_1)^2.$$

Read off:

$$v_1 = (1, 1), \quad c_1 = \frac{1}{2}; \quad v_2 = (1, -1), \quad c_2 = -\frac{1}{2}.$$

Translation:

$$u_{12} = \frac{1}{2} T_p(x; e_1 + e_2) - \frac{1}{2} T_p(x; e_1 - e_2).$$

This is the classical *polarization identity* for $p = 2$. Two real directions.

u_{112} : rank 3, three complex directions

Active exponents $(a_0, a_1) = (1, 2)$ with a_0 on the lighter variable. CCG rank $= a_1 + 1 = 3$.

Roots of unity: $\zeta \in \{1, \omega, \omega^2\}$, $\omega = e^{2\pi i/3}$.

$$v_\zeta = e_{i_0} + \zeta e_{i_1}, \quad c_\zeta = \frac{2}{3} \zeta.$$

Verification by direct expansion: $\sum_\zeta c_\zeta (v_\zeta \cdot w)^p$ has $w_0 w_1^2$ -coefficient $\frac{2}{3} \binom{3}{2} \sum_\zeta \zeta \cdot \zeta^2 = \frac{2}{3} \cdot 3 \cdot 3 = 6 = p!$, while every other monomial coefficient vanishes by the *roots-of-unity filter* $\sum_{\zeta^m=1} \zeta^k = m \cdot \mathbf{1}_{m|k}$.

u_{112} : rank 3, three complex directions

Active exponents $(a_0, a_1) = (1, 2)$ with a_0 on the lighter variable. CCG rank $= a_1 + 1 = 3$.

Roots of unity: $\zeta \in \{1, \omega, \omega^2\}$, $\omega = e^{2\pi i/3}$.

$$v_\zeta = e_{i_0} + \zeta e_{i_1}, \quad c_\zeta = \frac{2}{3} \zeta.$$

Verification by direct expansion: $\sum_\zeta c_\zeta (v_\zeta \cdot w)^p$ has $w_0 w_1^2$ -coefficient $\frac{2}{3} \binom{3}{2} \sum_\zeta \zeta \cdot \zeta^2 = \frac{2}{3} \cdot 3 \cdot 3 = 6 = p!$, while every other monomial coefficient vanishes by the *roots-of-unity filter* $\sum_{\zeta^m=1} \zeta^k = m \cdot \mathbf{1}_{m|k}$.

3 complex directions = 3 real polarization directions. Same length here: $a_0 = 1$, so $R_{\mathbb{R}} = R$ (CKOV 2017).

u_{1122} : rank 3 vs polarization 4

Active pattern $(2, 2)$, $p = 4$. Now $a_0 = 2$, the gap appears.

CCG rank $= a_1 + 1 = 3$. Roots of unity: $\zeta \in \{1, \omega, \omega^2\}$, $\omega = e^{2\pi i/3}$.

$$v_\zeta = e_1 + \zeta e_2, \quad c_\zeta = \frac{4}{3} \zeta.$$

u_{1122} : rank 3 vs polarization 4

Active pattern $(2, 2)$, $p = 4$. Now $a_0 = 2$, the gap appears.

CCG rank $= a_1 + 1 = 3$. Roots of unity: $\zeta \in \{1, \omega, \omega^2\}$, $\omega = e^{2\pi i/3}$.

$$v_\zeta = e_1 + \zeta e_2, \quad c_\zeta = \frac{4}{3} \zeta.$$

Real polarization needs *four* antipodal-merged directions (binary monomial: $R_{\mathbb{R}} = a + b = 2 + 2 = 4$).

Three complex directions instead of four real ones

$$u_{1122} = \frac{4}{3} \sum_{k=0}^2 \omega^k T_p(x; e_1 + \omega^k e_2).$$

This is where the algebra starts paying off.

u_{112233} : rank 9 vs polarization 13

Active pattern $(2, 2, 2)$, $p = 6$. CCG rank $= 3 \cdot 3 = 9$.

Polarization (with antipodal merging) needs 13 directions. Han–Moon real upper bound is also 13 here.

Roots of unity: ζ_1, ζ_2 each over 3rd roots of unity (9 pairs total).

$$v_{\zeta_1, \zeta_2} = e_1 + \zeta_1 e_2 + \zeta_2 e_3,$$

$$c_{\zeta_1, \zeta_2} = \frac{8}{9} \zeta_1 \zeta_2.$$

Total directions: $\prod_{j \geq 1} (a_j + 1)$.

Generic recipe (CCG construction).

- Pick *minimum-exponent* variable as base.
- For each other active j , pick ζ_j over $(a_j + 1)$ -th roots of unity.
- $c_\zeta = \frac{\alpha!}{\prod_j (a_j + 1)} \prod_j \zeta_j$.

Pattern table

pattern	example	R	$R_{\mathbb{R}}$ (best known)	polarization
(p)	$u_{1\dots 1}$	1	1	$\leq 2^{p-1}$
$(2, 1)$	u_{112}	3	3	3
$(1, 1, 1)$	u_{123}	4	4	4
$(3, 1)$	u_{1112}	4	4	—
$(2, 2)$	u_{1122}	3	4	4
$(2, 1, 1)$	u_{1123}	6	6	6
$(1, 1, 1, 1)$	u_{1234}	8	8	8
$(4, 2)$	u_{111122}	5	≤ 7	7
$(2, 2, 2)$	u_{112233}	9	≤ 13	13
(1^6)	u_{123456}	32	32	32

Bold red: where the complex Waring schedule is strictly shorter. Sweet spot: *repeated, balanced* active exponents.

Square-free (1^p) saturates 2^{p-1} over both $\mathbb{R}$ and — the method offers no rank advantage there.

The two operating regimes

Direct nested autodiff (B1)

- 1 One forward pass.
- 2 p calls of `torch.autograd.grad` with `create_graph=True`.
- 3 Graph after k -th call has depth $\sim kL$;
($k + 1$)-st call *replays* entire graph.

Total replay cost $\sum_{k=1}^p k \cdot C_{\text{fwd}} = \Theta(p^2 C_{\text{fwd}})$ plus
 $\Theta(pL)$ autograd dispatch ops.

Waring + Taylor jet (B2/B3/B4)

- 1 Pick R directions v_r (Waring or polarization).
- 2 Replace each layer by a *jet rule*: each tensor becomes a length- $(p + 1)$ tuple of Taylor coefficients.
- 3 One forward pass per direction; R passes total.

Total cost $\Theta(R \cdot L \cdot ((p+1)C_{\text{fwd}} + BH p^2))$ with *no* autograd graph chaining.

Where the speedup comes from — four levers

Lever 1. Forward-mode beats reverse-mode for “one number out”.

Reverse-mode cost scales with output dimension. Here output is one scalar; forward jet through $p + 1$ Taylor coefficients is more direct.

Where the speedup comes from — four levers

Lever 1. Forward-mode beats reverse-mode for “one number out”.

Reverse-mode cost scales with output dimension. Here output is one scalar; forward jet through $p + 1$ Taylor coefficients is more direct.

Lever 2. No graph chaining.

p nested grad calls build a graph PyTorch must traverse twice (build + execute). Constants are large: each autograd op carries $\sim 10\text{--}100\ \mu\text{s}$ of dispatch overhead.

Where the speedup comes from — four levers

Lever 1. Forward-mode beats reverse-mode for “one number out”.

Reverse-mode cost scales with output dimension. Here output is one scalar; forward jet through $p + 1$ Taylor coefficients is more direct.

Lever 2. No graph chaining.

p nested grad calls build a graph PyTorch must traverse twice (build + execute). Constants are large: each autograd op carries $\sim 10\text{--}100\ \mu\text{s}$ of dispatch overhead.

Lever 3. One GEMM per Linear layer per pass.

Stack the $p + 1$ jet terms along the batch axis; one Linear becomes one $(B(p+1)) \times H \times H$ matmul instead of $p + 1$ separate ones.

Where the speedup comes from — four levers

Lever 1. Forward-mode beats reverse-mode for “one number out”.

Reverse-mode cost scales with output dimension. Here output is one scalar; forward jet through $p + 1$ Taylor coefficients is more direct.

Lever 2. No graph chaining.

p nested grad calls build a graph PyTorch must traverse twice (build + execute). Constants are large: each autograd op carries $\sim 10\text{--}100\ \mu\text{s}$ of dispatch overhead.

Lever 3. One GEMM per Linear layer per pass.

Stack the $p + 1$ jet terms along the batch axis; one Linear becomes one $(B(p+1)) \times H \times H$ matmul instead of $p + 1$ separate ones.

Lever 4. Algebraic rank reduction.

Replace the implicit p -fold tensor probe of nested AD by $R \leq R_{(w^\alpha)}$ directional probes. For repeated patterns $R \ll 2^{p-1} \leq p!$.

Order-of-magnitude analysis

Let $C_{\text{fwd}} = \Theta(LBH^2)$ be one forward pass cost, κ the autograd per-op dispatch constant, H width, L depth, p order.

	<i>Direct nested AD</i>	<i>Waring + Taylor jet</i>
asymptotic time	$\Theta(p^2 C_{\text{fwd}} + p L \kappa)$	$\Theta(R (p C_{\text{fwd}} + LBHp^2))$
memory tensors	$\Theta((p+1)L)$ <i>plus graph</i>	$\Theta((p+1)L)$, <i>no graph</i>
backward to θ	extra graph replay	one custom VJP per activation
rank-aware?	no	yes (uses $R_{(\alpha)}$)

Order-of-magnitude analysis

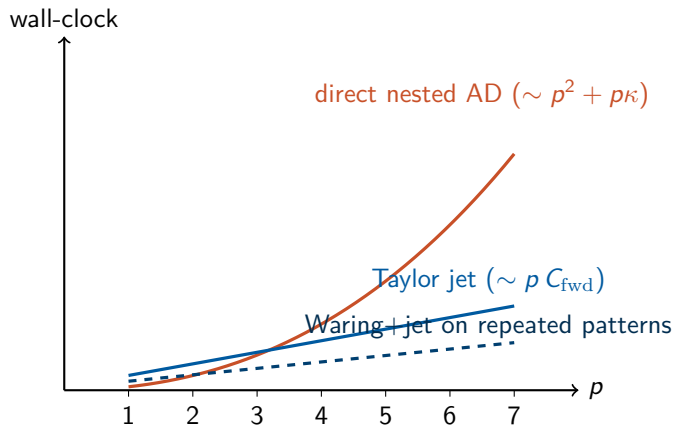
Let $C_{\text{fwd}} = \Theta(LBH^2)$ be one forward pass cost, κ the autograd per-op dispatch constant, H width, L depth, p order.

	<i>Direct nested AD</i>	<i>Waring + Taylor jet</i>
asymptotic time	$\Theta(p^2 C_{\text{fwd}} + p L \kappa)$	$\Theta(R(p C_{\text{fwd}} + LBHp^2))$
memory tensors	$\Theta((p+1)L)$ <i>plus graph</i>	$\Theta((p+1)L)$, <i>no graph</i>
backward to θ	extra graph replay	one custom VJP per activation
rank-aware?	no	yes (uses $R_{(\alpha)}$)

Two qualitative facts.

- For *any* pattern α , the jet path removes the $p L \kappa$ dispatch term — already a multiplicative speedup on small networks at $p = 6$.
- For *repeated-index* patterns, R shrinks by a constant factor (rank-vs-polarization) on top of that.

Predicted vs naive growth



Up to constants determined by L, H, B , the gap widens with p — exactly because the per-step cost of nested AD is super-linear in p while the jet path's per-direction cost grows only linearly in p .

One forward pass, all $p + 1$ Taylor coefficients

Each tensor y becomes a length- $(p + 1)$ tuple $(y_0, y_1, \dots, y_p)$, where $y_k = \frac{1}{k!} d^k y / d\tau^k$ at $\tau = 0$.

Jet rules used in this work:

- **Linear $y = Wx + b$:** $y_0 = Wx_0 + b$, $y_k = Wx_k$ for $k \geq 1$. Stack as one big GEMM.
- **sinh:** auxiliary $z = \cosh(x)$, then for $k \geq 1$

$$y_k = \frac{1}{k} \sum_{j=0}^{k-1} (k-j) z_j x_{k-j}, \quad z_k = \frac{1}{k} \sum_{j=0}^{k-1} (k-j) y_j x_{k-j}.$$

- Similar coupled recurrences for $\tanh$, σ , $\sin$.

Closed-form custom VJP per activation $g_x^{(m)} = \sum_{j=0}^{p-m} \overline{q_j} g_y^{(m+j)} \Rightarrow$ no nested autograd in backward.

Why sinh for the PINN setup

- Entire on : every Taylor jet is well-defined globally. Compatible with *complex* Waring directions $v_\zeta \in \mathbb{C}^d$ without any analytic continuation worry.
- Unbounded range: better at representing solutions with non-saturating gradients than $\tanh$.
- Same algebraic recurrence skeleton as $\sin/\cos$, but with no sign flip (use $\cosh$ in place of $\cos$, drop the minus).
- Pairs naturally with *complex-parameter* MLPs: weights $W \in \mathbb{C}^{m \times n}$ multiply complex jet coefficients without runtime casting.

The full backend

- ❶ Read multi-index α . Sort active exponents $1 \leq a_0 \leq \dots \leq a_n$.
- ❷ Compute $R_{(\alpha)=\prod_{j \geq 1} (a_j+1)}$ and the polarization count.
- ❸ Generate directions $\{v_r, c_r\}$:
 - pure power ($n = 0$): $v = e_{i_0}$, $c = p!$.
 - known-rank pattern ($a_0 = 1$ or binary): real Waring formula.
 - repeated balanced and $R_{\ll}$ polarization: complex roots of unity.
 - otherwise: real polarization fallback.
- ❹ Forward Taylor jet of order p on each direction.
- ❺ Sum: $\partial^\alpha u(x) = \sum_r c_r (T_p\text{-jet})_r$.

End-user API:

```
from apolarity import single_monomial_partial
d_alpha = single_monomial_partial(model, x, alpha,
                                   backend="auto")
```

Dispatch policy

$$\text{backend}(\alpha) = \begin{cases} \text{pure_power_rank1} & \text{if } n = 0, \\ \text{waring_complex_jet} & \text{if } R_{(\alpha)} \leq 0.8 \# \text{pol}(\alpha), \\ \text{waring_real_jet} & \text{if } a_0 = 1 \text{ or binary pattern,} \\ \text{polarization_jet} & \text{otherwise.} \end{cases}$$

In *backward mode* (parameter-gradient training), prefer real polarization to avoid the constant overhead of complex `torch.autograd`.

Realification fallback (Chebyshev nodes + scale ρ search) is available for patterns currently lacking a closed-form real construction.

Take-aways

- Computing one mixed partial $\partial^\alpha u$ is the same as extracting one coefficient of a homogeneous polynomial.
- That extraction problem is, exactly, a Waring decomposition of the monomial w^α in dual variables.
- Carlini–Catalisano–Geramita gives the optimal direction count $R = \prod_{j \geq 1} (a_j + 1)$, and an explicit roots-of-unity construction realizes it.
- Plugged into a Taylor-jet forward pass through Linear/sinh MLPs (real or complex parameters), this avoids the autograd-graph chaining of nested grad and reduces per-deriv wall-clock by an order of magnitude at $p = 6$.
- Best regime: *repeated, balanced* multi-indices — exactly what fourth- and sixth-order PDE residuals in PINN demand.
- Worst regime: *square-free*, where we tie polarization.

Take-aways

- Computing one mixed partial $\partial^\alpha u$ is the same as extracting one coefficient of a homogeneous polynomial.
- That extraction problem is, exactly, a Waring decomposition of the monomial w^α in dual variables.
- Carlini–Catalisano–Geramita gives the optimal direction count $R=\prod_{j\geq 1}(a_j+1)$, and an explicit roots-of-unity construction realizes it.
- Plugged into a Taylor-jet forward pass through Linear/sinh MLPs (real or complex parameters), this avoids the autograd-graph chaining of nested grad and reduces per-deriv wall-clock by an order of magnitude at $p = 6$.
- Best regime: *repeated, balanced* multi-indices — exactly what fourth- and sixth-order PDE residuals in PINN demand.
- Worst regime: *square-free*, where we tie polarization.

The exact deterministic backend complements stochastic estimators like STDE (Shi–Hu–Lin–Kawaguchi, NeurIPS 2024): it gives machine-precision exactness for one fixed multi-index, where stochastic methods give bias-free expectation but residual variance.

Apolarity.

Algebraic geometry of monomials

→ optimal directions for high-order partial derivatives

→ deterministic exact PINN backend.